



Name: Ayush Chaubey

PRN: 25070122058

Division: A2

ASSIGNMENT No: 10

TITLE: Write a Java program to demonstrate the use of abstract classes and abstract methods

Problem Statement: Write a Java program to demonstrate the use of abstract classes and abstract methods.

Learning Objectives: To implement abstraction using abstract classes and abstract methods.

Theory: Abstract classes provide partial implementation of a program and cannot be instantiated directly. They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

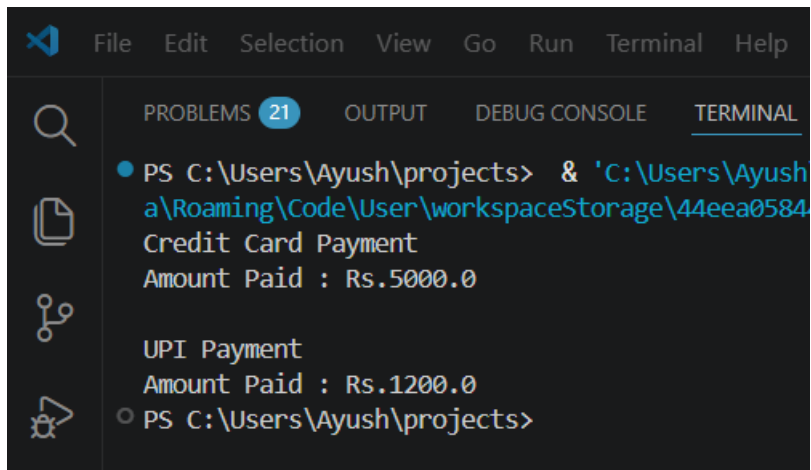
Exercise:

1. Create an abstract Payment class and implement Credit Card and UPI payment classes.

Program:

```
helloworld > J Main10.java 2 ●
1  abstract class Payment
2  {
3      abstract void pay(double amount);
4  }
5  class CreditCard extends Payment
6  {
7      void pay(double amount)
8      {
9          System.out.println(x: "Credit Card Payment");
10         System.out.println("Amount Paid : Rs." + amount);
11     }
12 }
13 class UPI extends Payment
14 {
15     void pay(double amount)
16     {
17         System.out.println(x: "UPI Payment");
18         System.out.println("Amount Paid : Rs." + amount);
19     }
20 }
21 public class Main10
22 {
23     Run main | Debug main | Run | Debug
24     public static void main(String[] args)
25     {
26         Payment p1 = new CreditCard();
27         Payment p2 = new UPI();
28
29         p1.pay(amount: 5000);
30         System.out.println();
31
32         p2.pay(amount: 1200);
33     }
34 }
```

Output:

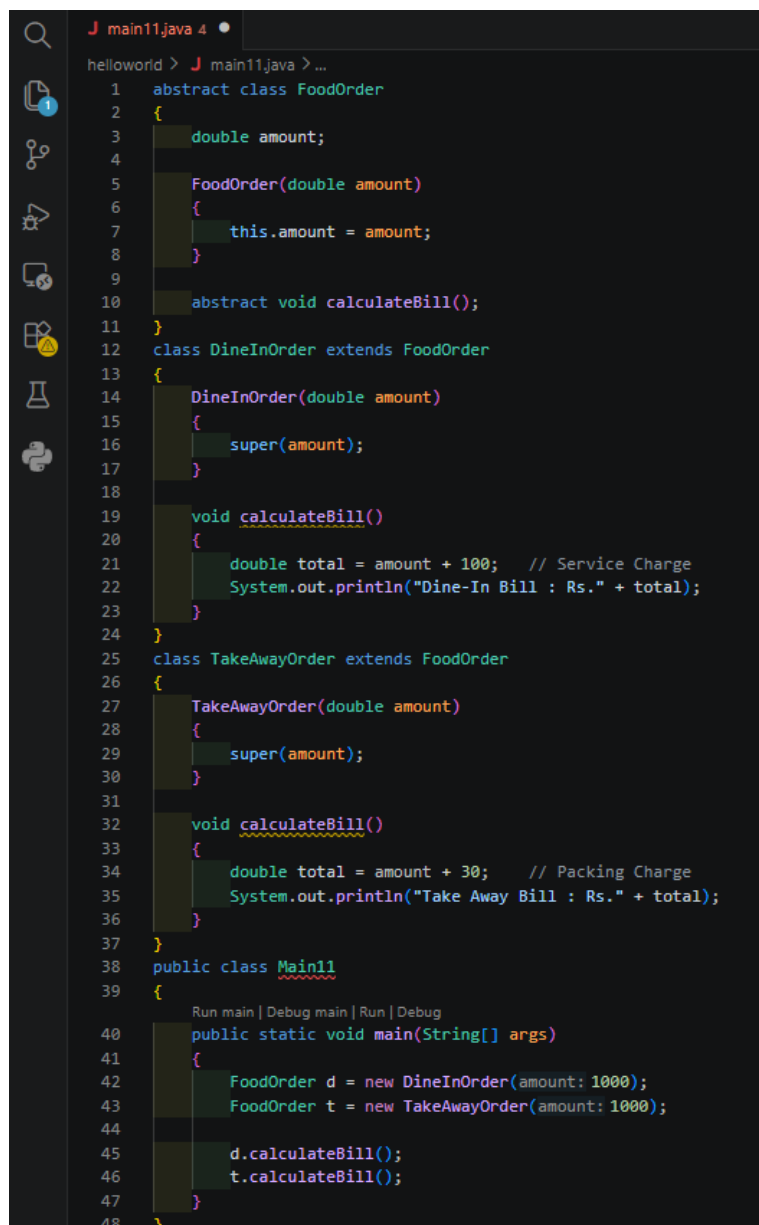


```
PS C:\Users\Ayush\projects> & 'C:\Users\Ayush\Roaming\Code\User\workspaceStorage\44eea0584'
Credit Card Payment
Amount Paid : Rs.5000.0

UPI Payment
Amount Paid : Rs.1200.0
PS C:\Users\Ayush\projects>
```

2. Create an abstract FoodOrder class with an abstract method calculateBill(). Implement two subclasses, DineInOrder and TakeAwayOrder, to calculate and display the total bill based on the order type.

Program:



```
J main11.java 4 •
helloworld > J main11.java > ...
1  abstract class FoodOrder
2  {
3      double amount;
4
5      FoodOrder(double amount)
6      {
7          this.amount = amount;
8      }
9
10     abstract void calculateBill();
11 }
12 class DineInOrder extends FoodOrder
13 {
14     DineInOrder(double amount)
15     {
16         super(amount);
17     }
18
19     void calculateBill()
20     {
21         double total = amount + 100; // Service Charge
22         System.out.println("Dine-In Bill : Rs." + total);
23     }
24 }
25 class TakeAwayOrder extends FoodOrder
26 {
27     TakeAwayOrder(double amount)
28     {
29         super(amount);
30     }
31
32     void calculateBill()
33     {
34         double total = amount + 30; // Packing Charge
35         System.out.println("Take Away Bill : Rs." + total);
36     }
37 }
38 public class Main1
39 {
40     Run main | Debug main | Run | Debug
41     public static void main(String[] args)
42     {
43         FoodOrder d = new DineInOrder(amount: 1000);
44         FoodOrder t = new TakeAwayOrder(amount: 1000);
45
46         d.calculateBill();
47         t.calculateBill();
48     }
49 }
```

Output:

Dine-In Bill : Rs.1100.0

Take Away Bill : Rs.1030.0